Supervised Learning: From Linear Models to Neural Networks

Machine Learning: The Big Picture

Machine Learning refers to algorithms that improve through experience by learning patterns from data rather than relying on explicit programming.

Given	Goal
Data (examples, observations, interactions)	Learn patterns to make predictions or decisions

Reinforcement Learning: Learn from rewards and penalties

- Agent takes actions and receives feedback from environment
- Examples: game playing (chess, Go), robotics, autonomous driving

Already Introduced This!

Unsupervised Learning: Find patterns in unlabeled data

- Data has inputs but no labeled outputs
- Examples: clustering customers, dimensionality reduction, anomaly detection

Presented in 272.

Supervised Learning: Learn from labeled examples

- Training data includes correct answers (labels)
- Examples: spam detection, image classification, price prediction

Our focus for the rest of the course.

Supervised Learning: The Setup

Training Data	(input, output) pairs	$e_1 = [x_{1_1}, x_{2_1}, \dots, x_{n_1}] \rightarrow y_1$ $e_2 = [x_{1_2}, x_{2_2}, \dots, x_{n_2}] \rightarrow y_2$ … $e_m = [x_{1_m}, x_{2_m}, \dots, x_{n_m}] \rightarrow y_m$ $e = (\mathbf{x}, y)$ where $\mathbf{x} \in \mathbb{R}^n$ and $\boxed{y \in \mathbb{R}}$	(describes regression, not classification)
Learning Algorithm	Finds patterns	Find the best parameters θ by minimizing error:	$\min_{\theta} \sum_{i=1}^{m} (h(\mathbf{x}_i) - y_i)^2$
Hypothesis: h	**Learned Model** hypothesis h about the world; it approximates the true function.	Map feature vectors to predictions:	$h: \mathbb{R}^n \rightarrow \mathbb{R}$
Inference (Prediction)	Apply h for inference on new inputs:	$\hat{y}$ for new example $\mathbf{x}^*$	$\hat{y} = h(\mathbf{x}^*)$

example feature−vector output

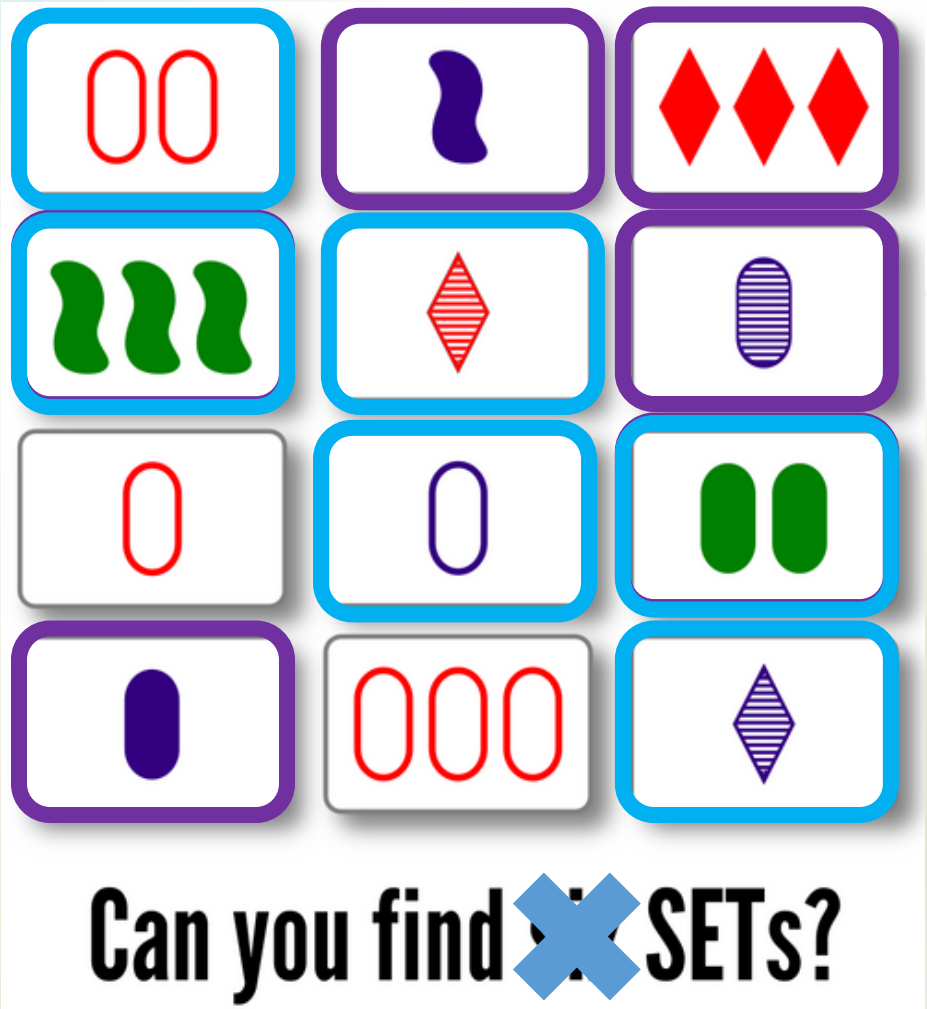
* Refers to a new example

Supervised Classification Example: Set

Features Color, Shape, Number, Shading

Task Classify whether 3 cards form a valid Set according to the rules of Set:

- **"Same trait":** All three cards share the exact same version of a trait
- **"All different"**: Each of the three cards possesses a unique version of a trait, with no repeats



Supervised Classification Example: Set

Scenario

Assume the machine does not know the rules of Set. That is, it must intuit the pattern from examples (i.e., no explicit programming of "all same or all different").

Label $y \in \{\boldsymbol{Set}, \boldsymbol{Not\ Set}\}$

Training Data

[[red, oval, 1, solid], [red, diamond, 1, solid], [red, squiggle, 1, solid]] ***Set***

[[red, oval, 1, solid], [green, oval, 2, striped], [purple, oval, 3, empty]] ***Set***

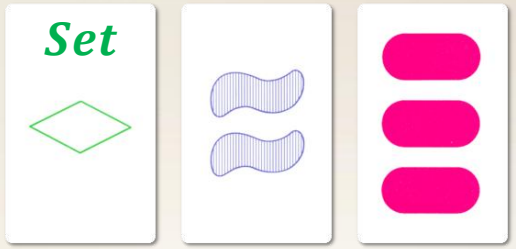
[[red, oval, 1, solid], [red, oval, 2, striped], [green, diamond, 1, solid]] ***Not Set***

... (many more labeled examples)

Learning Find h that predicts whether a combination of 3 cards is a Set.

Inference Given 3 new cards* → predict ***Set*** or ***Not Set***

* indicates a new input vector

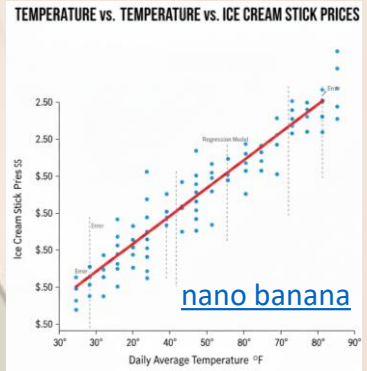


Machine Learning: We provide examples, algorithm learns 'rules'.

(Two) Types of Supervised Learning

The difference is the type of output we are predicting.

Core Question	How much?	Which one?
Type	**Regression**	**Classification**
Predicts	Predict continuous values	Predict discrete categories
Output	$y \in \mathbb{R}$	$y \in \{1,2,\dots,k\}$ (class labels)
Examples	• house prices • temperature • stock prices • blood pressure	• spam/not spam • cat/dog • disease/healthy • Categorizing a product (electronics, clothing, etc.)



Supervised Regression Example: Temperature

Task Predict temperature from time and weather conditions

Training Data

$\mathbf{x} = [\text{hour}, \text{cloud_cover}, \text{humidity}] \in \mathbb{R}^3$

$y \in \mathbb{R}$ (in °C)

$\mathbf{e_1}$ = [6am, 20%, 80%] → 15°C

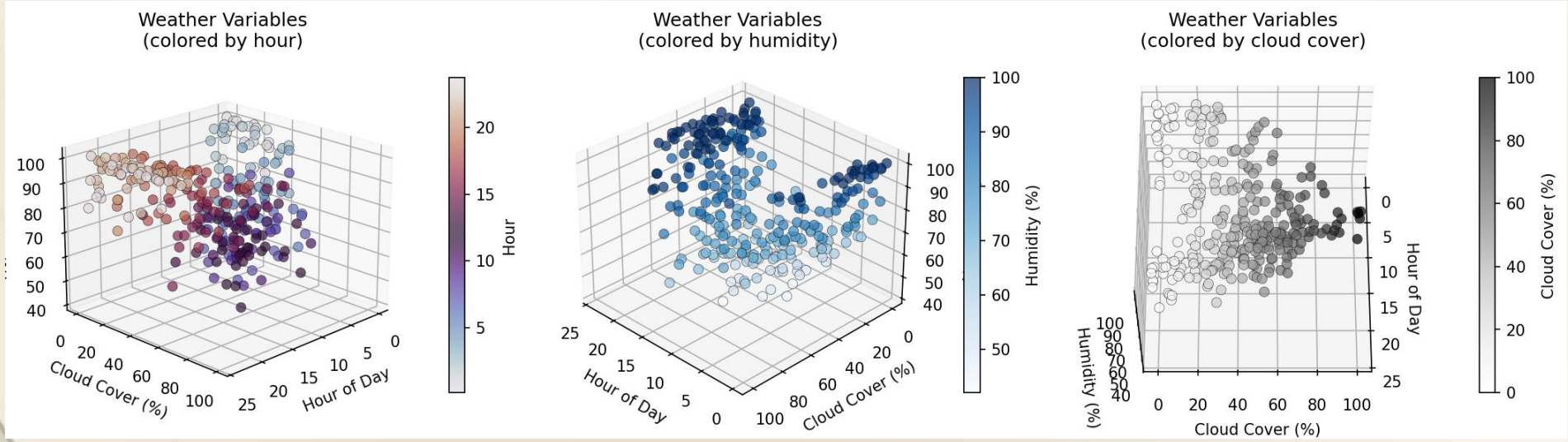
$\mathbf{e_2}$ = [2pm, 10%, 40%] → 28°C

$\mathbf{e_3}$ = [9pm, 60%, 70%] → 18°C

…

Learning Find h that predicts temperature

Inference Given new conditions $\mathbf{x}^*$ → predict temperature $\hat{y}$



Observe, we are visualizing the input points only.

Training, Tuning, and Testing Pipeline and Data

60-80%	10-20%	10-20%

Training Phase

Use training set to learn parameters θ; minimize loss:

$$\min_{\theta} \sum \left(y_i - h(y_i)\right)^2$$

Learn model parameters (60-80%):

$e_1 = (\mathbf{x_1}, y_1)$,

$e_2 = (\mathbf{x_2}, y_2)$,

…,

$e_n = (\mathbf{x_n}, y_n)$

Tuning/Validation Phase

Evaluate hyperparameter (λ) combinations on validation set. Select the best model configuration ($\lambda^* = \underset{\lambda}{\operatorname{argmin}}\, validation_error$)

Select hyperparameters (λ) (10-20%):

- Choose model complexity, regularization, etc.
- Never used for learning parameters

Testing Phase

Final evaluation on test set

Report generalization performance

Final evaluation (10-20%):

- Never used during training or tuning
- Measures generalization to new data

Each phase has a specific role and dataset because we want to prevent overfitting to our training data (where the model memorizes the training data and generalizes poorly).

The Generalization Challenge

Underfitting

Model too simple

Poor on both training and test sets

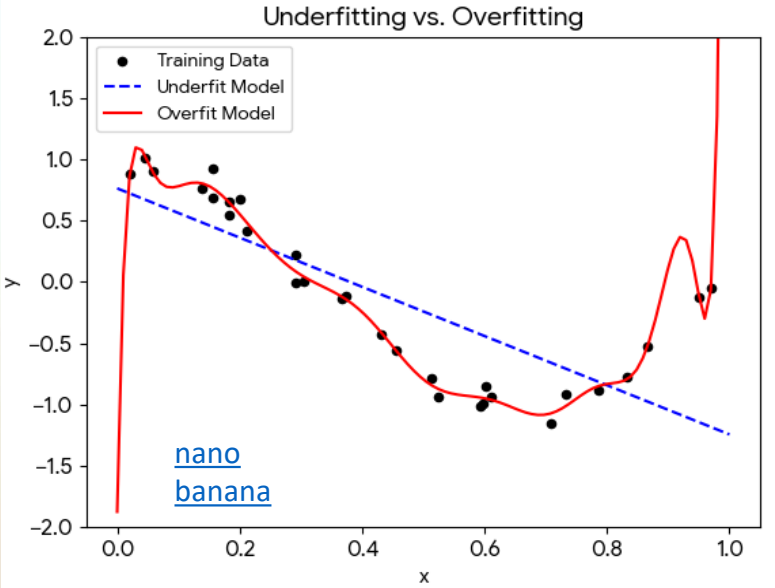
Does not capture patterns in data

Overfitting

Model memorizes training data

Perfect on training set, poor on test set

Too complex for the amount of data



Goal

Balance complexity with generalization

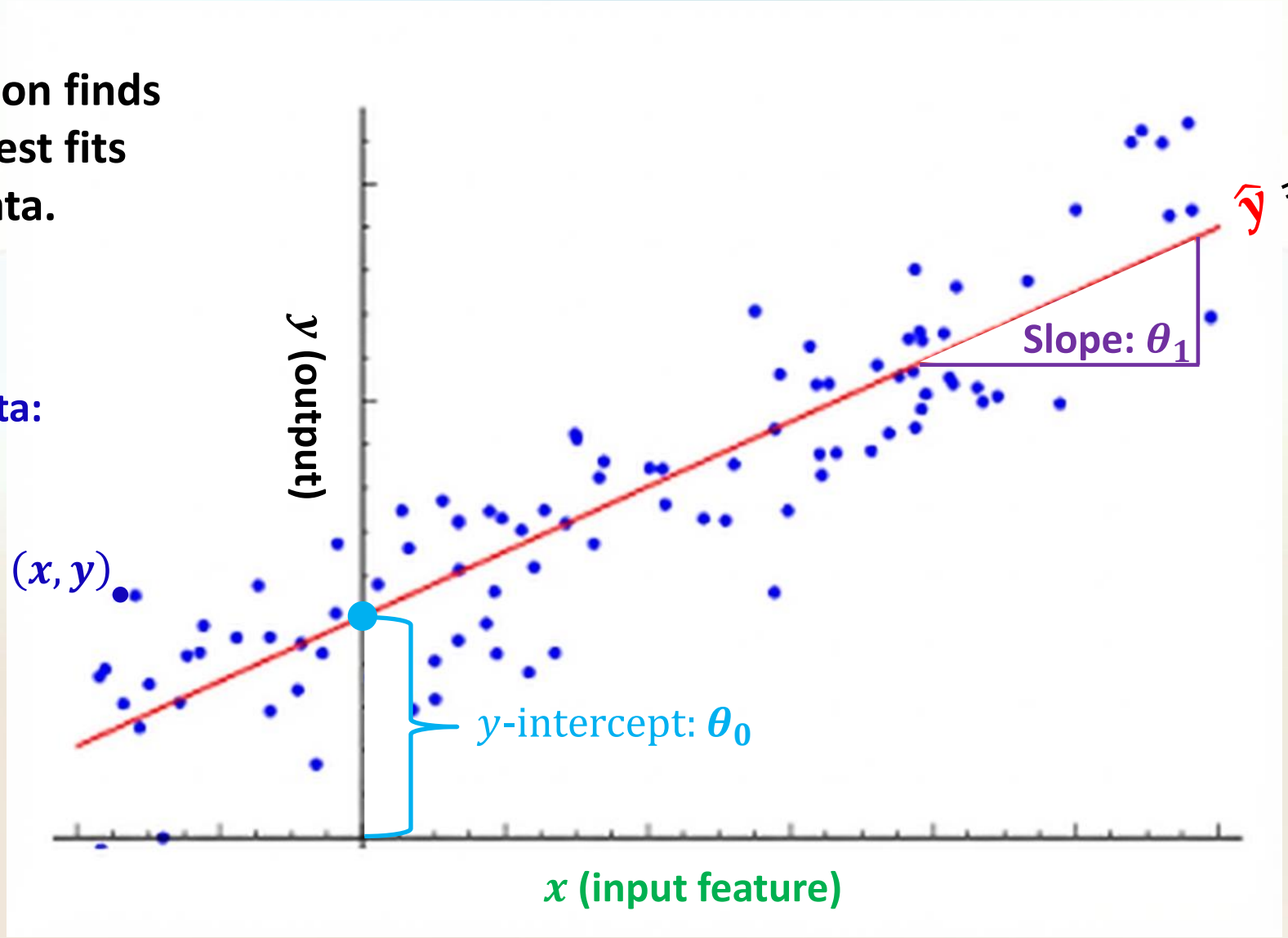
Good training performance **and** good test performance

Linear Regression

Univariate Linear Regression

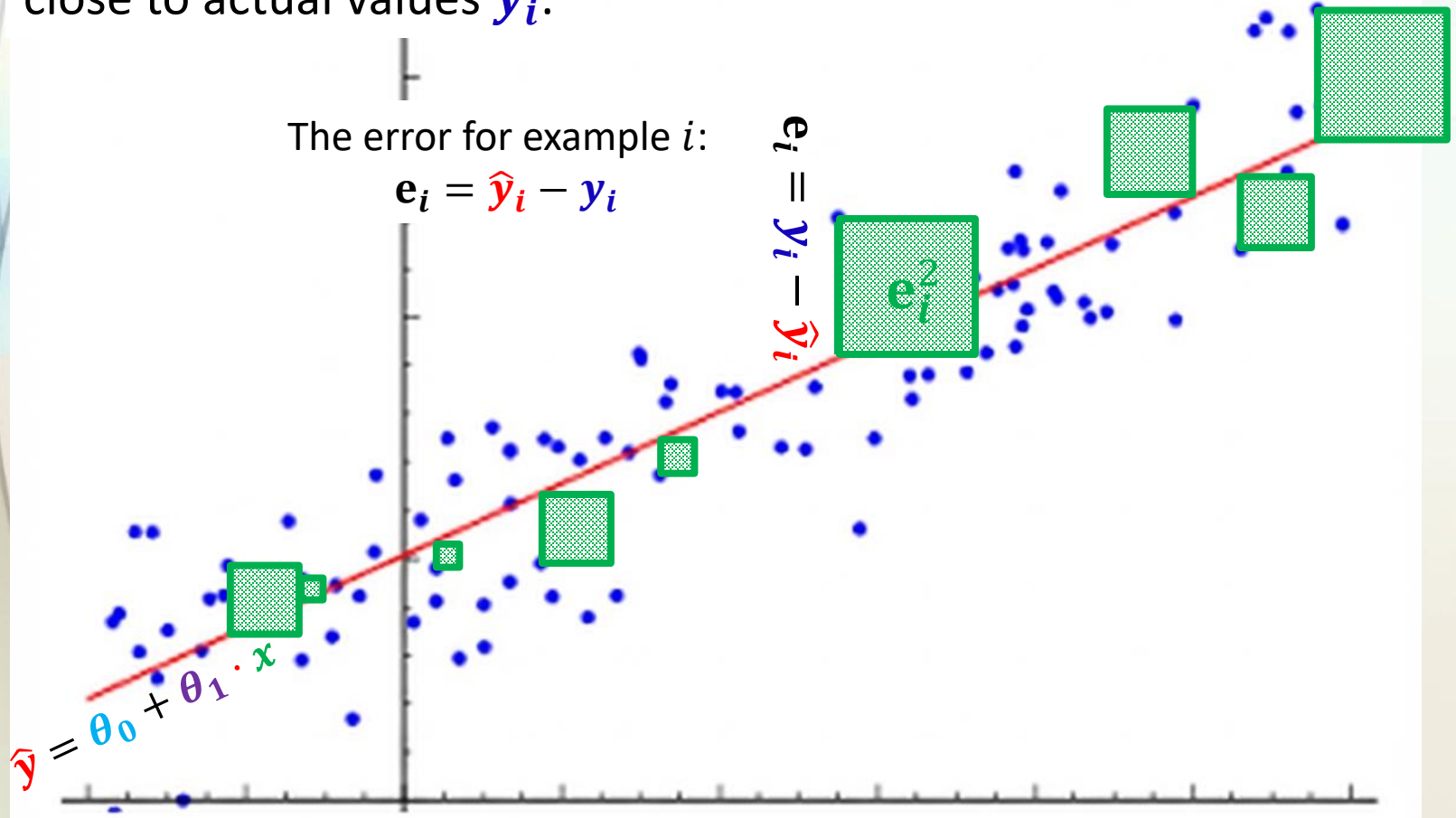
Linear regression finds the line that best fits the training data.

Training data:
(x_i, y_i)



Univariate Linear Regression: MSE

We want predictions $\hat{y}_i$ close to actual values y_i.



The squared error for example i:

$$\mathbf{e}_i^2 = (\hat{y}_i - y_i)^2$$

- Always positive
- Penalizes large errors more heavily

Mean Squared Error (MSE):

$$\text{MSE} = \frac{1}{m} \min_{\theta_0, \theta_1} \sum_{i=1}^{m} (\hat{y}_i - y_i)^2$$

Our objective is to find θ_0 and θ_1 that minimize MSE.

Multivariate Linear Regression

Input $\mathbf{x} = [x_1, x_2, \dots, x_n] \in \mathbb{R}^n$

$x_1, x_2, \dots, x_n$: feature values (e.g., size, bedrooms, age)

Parameters $\boldsymbol{\theta} = [\theta_0, \theta_1, \theta_2, \dots, \theta_n] \in \mathbb{R}^{n+1}$

θ_0: intercept

$\theta_1, \theta_2, \dots, \theta_n$: weights for each feature

Model

$$\hat{y} = h_{\boldsymbol{\theta}}(\mathbf{x}) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n = \boldsymbol{\theta} \cdot \mathbf{x}$$

($\mathbf{x}$ is augmented with $x_0 = 1$)

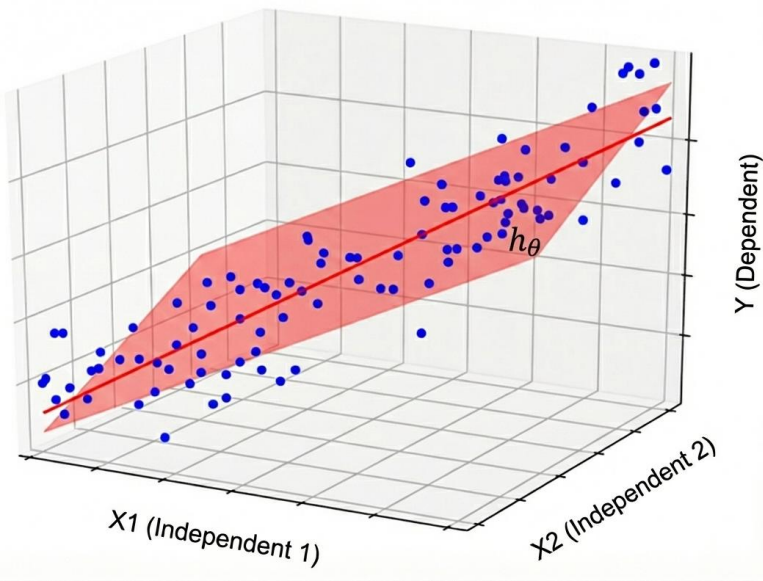
Training Find $\boldsymbol{\theta}$ that minimizes MSE:

$$\frac{1}{m} \min_{\boldsymbol{\theta}} \sum_{i=1}^{m} (\hat{y}_i - y_i)^2$$

(Same principle as univariate)

Result Inference: $h_{\boldsymbol{\theta}}: \mathbb{R}^{n+1} \to \mathbb{R}$

Maps feature vectors to predictions using learned $\boldsymbol{\theta}$

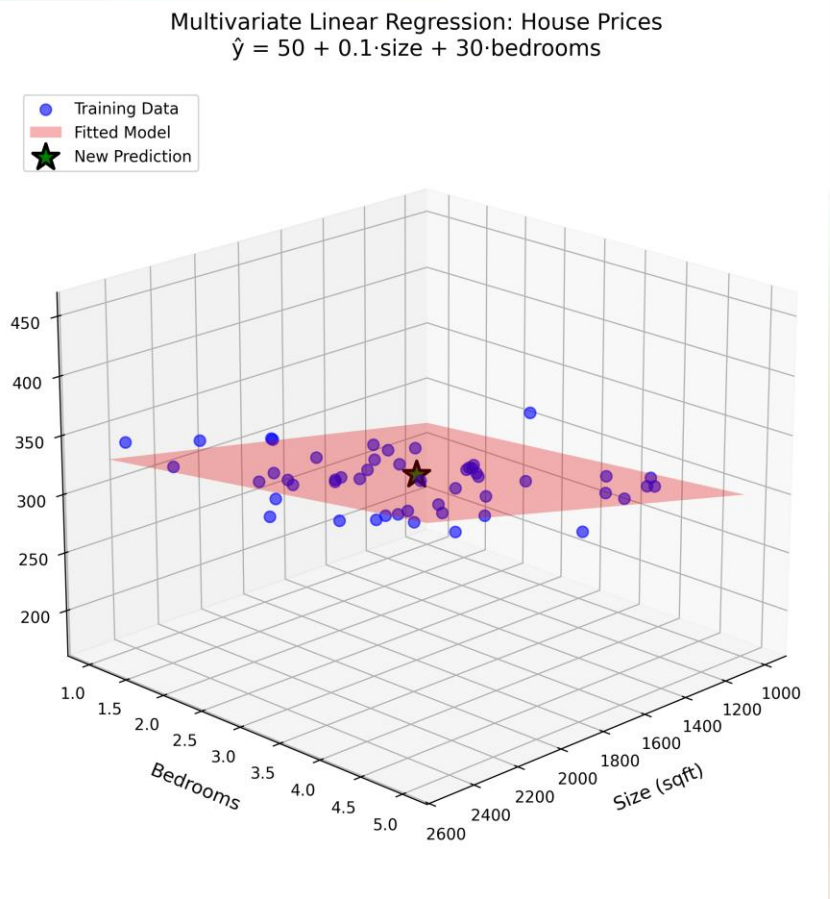


Example: Predicting House Prices

Features x_1: size (in ft^2) x_2: number of bedrooms

Training Data

Size (sqft)	Bedrooms	Actual Price	
1561	3	$309K	
1233	2	$249K	
1030	4	$285K	
1275	4	$309K	
1917	4	$355K	
2177	4	$388K	
1800	**3**	**?**	**$320K ★**



Model $\hat{y} = 50 + 0.1x_1 + 30x_2$

- $\theta_0 = 50$: Base price ($50K)
- $\theta_1 = 0.1$: Each ft^2 adds $100
- $\theta_2 = 30$: Each bedroom adds $30K

Inference Example $\hat{y} = h(1800, 3) = 50 + 0.1 \cdot 1800 + 30 \cdot 3 = 320\text{K}$

Example: Email Spam Detection

Features $x_1 = freq("free")$ $x_2 = freq("money")$

Training Data

$x_1 = freq("free")$	$x_2 = freq("money")$	$\hat{y}$
0.10	0.05	Not Spam
0.80	0.70	Spam
0.05	0.10	Not Spam
0.90	0.60	Spam
0.15	0.20	Not Spam
0.65	0.85	Spam

Model

$$\hat{y} = 0.16 + 0.61x_1 + 0.54x_2$$

- $\theta_0 = 0.16$: Base prediction
- $\theta_1 = 0.61$: Weight for "free" frequency
- $\theta_2 = 0.54$: Weight for "money" frequency

Inference Example

$$\hat{y} = h(freq("\text{free}") = 0.70, freq("\text{money}") = 0.60) = 0.16 + 0.61 \cdot 0.70 + 0.54 \cdot 0.60 = 0.911$$

Problem

- Real-valued output does not directly correspond to discrete classes
- Linear regression produces a real number (0.91); we need ***Spam*** or ***Not Spam***
- Predictions are unbounded and can be < 0 or > 1
- We want to interpret the result as a probability: $P(spam \mid \mathbf{x})$, not an arbitrary number

We need a better approach…
Logistic Regression

Training Linear Regression

$$\hat{y} = h_{\boldsymbol{\theta}}(\mathbf{x}) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n = \boldsymbol{\theta} \cdot \mathbf{x}$$

Our ***goal*** for training is to find parameters $\boldsymbol{\theta}$ that minimize prediction error.

Mean Squared Error as Loss function for Linear Regression

$$MSE(\boldsymbol{\theta}) = \frac{1}{m} \min_{\boldsymbol{\theta}} \sum_{i=1}^{m} (h_{\boldsymbol{\theta}}(\mathbf{x}_i) - \boldsymbol{y_i})^2$$

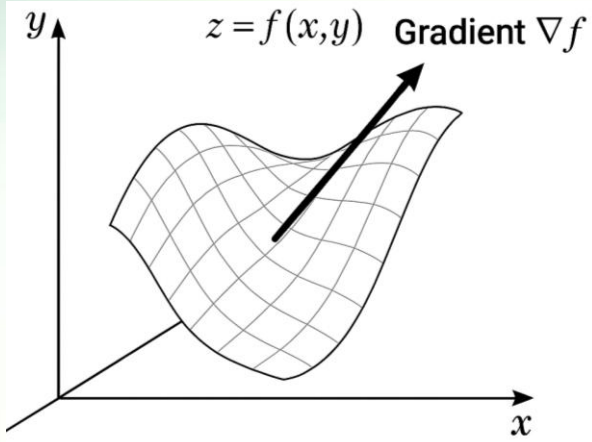
A ***Loss Function*** measures prediction error.

- Training attempts to minimize loss
- The lower the loss results in better predictions

How do we find the optimal parameters $\boldsymbol{\theta}^*$ that minimizes $MSE(\boldsymbol{\theta})$?

We will consider an iterative method called ***gradient descent***.

What is a Gradient?



Recall, our model has $n+1$ parameters $\boldsymbol{\theta} = [\theta_0, \theta_1, \theta_2, \dots, \theta_n] \in \mathbb{R}^{n+1}$.

How do we decide how to increase or decrease each parameter? Again, our goal is to reduce the loss, $MSE(\boldsymbol{\theta})$.

For a function f, the ***gradient*** ∇f is a mathematical construct that helps us identify the direction and rate of the steepest increase.

In a multivariable setting, ∇f is a vector of partial derivatives, where each component represents how sensitive the function is to changes in one variable

Thus, we are interested in computing $\nabla MSE(\boldsymbol{\theta})$.

$$\nabla MSE(\boldsymbol{\theta}) = \left[\frac{\partial MSE}{\partial \theta_0}, \frac{\partial MSE}{\partial \theta_1}, \frac{\partial MSE}{\partial \theta_2}, \dots, \frac{\partial MSE}{\partial \theta_n}\right]$$

In gradient descent, we move *opposite* the gradient (downhill) to minimize MSE.

The Optimization Problem

Now that we understand gradients, we can use them to solve our optimization problem. Gradients point uphill, so gradient descent moves downhill toward a minimum of our loss (error) function.

Training Data

Suppose our training data is defined as $\mathbf{X}_{m\times(n+1)}$ (examples × features) with corresponding true output vector $\mathbf{Y}_{m\times 1}$.

Each of the m rows of $\mathbf{X}_{m\times(n+1)}$ is a training example: $[1, x_1, x_2, \dots, x_n]$

Parameters

$$\boldsymbol{\theta} = [\theta_0, \theta_1, \theta_2, \dots, \theta_n] \in \mathbb{R}^{n+1}$$

Model

The linear system $\mathbf{X} \cdot \boldsymbol{\theta} = \mathbf{Y}$.

Loss Function

$$MSE(\boldsymbol{\theta}) = \frac{1}{m} \min_{\boldsymbol{\theta}} \sum_{i=1}^{m} (h_{\boldsymbol{\theta}}(\mathbf{x}_i) - \boldsymbol{y_i})^2$$

Gradient Descent (Iterative Optimization)

Initialize $\boldsymbol{\theta}$ randomly (or to zeros)

repeat until convergence:

- **for** all training examples $i = 1, \dots, m$, ***compute predictions***:
- **for** all training examples $i = 1, \dots, m$, ***compute errors***:
- Compute the direction of steepest increase (***gradient***):
- **Update all parameters** simultaneously in the direct of the steepest decrease:
- If MSE is decreasing by less than ϵ per iteration, stop (as the algorithm converged)

return $\boldsymbol{\theta}^*$

$$\hat{y}_i = h_{\boldsymbol{\theta}}(\mathbf{x}_i) = \boldsymbol{\theta}^{\mathbf{T}}\mathbf{x}_i$$

$$\mathbf{e}_i = \hat{\boldsymbol{y}}_i - \boldsymbol{y}_i$$

$$\nabla MSE(\boldsymbol{\theta}) = \frac{1}{m}\min_{\boldsymbol{\theta}} \sum_{i=1}^{m} \mathbf{e}_i \mathbf{x}_i$$

$$\boldsymbol{\theta} := \boldsymbol{\theta} - \alpha \cdot \nabla MSE(\boldsymbol{\theta})$$

α: learning rate that controls step size

Idea: Start with random $\boldsymbol{\theta}$, iteratively improve it.

Example: Single Gradient Descent Update I

Setup

One training example: $\mathbf{x} = [1,2]$, true label $y = 1$

Initial parameters: $\boldsymbol{\theta} = [0.5,0.2]$

Learning Rate: $\alpha = 0.1$

Step 1: Compute Prediction $\hat{y} = h_{\boldsymbol{\theta}}(\mathbf{x}_i) = \boldsymbol{\theta}^{\mathbf{T}}\mathbf{x}_i = 0.5 \cdot 1 + 0.2 \cdot 2 = 0.9$

Step 2: Compute Error $\mathbf{e} = [\hat{y} - \boldsymbol{y}] = [0.9 - 1] = [-0.1]$

Step 3: Compute Gradient $\nabla MSE(\boldsymbol{\theta}) = \mathbf{e} \cdot \mathbf{x} = -0.1 \cdot [1,2] = [-0.1, -0.2]$

Step 4: Update Parameters $\boldsymbol{\theta} := \boldsymbol{\theta} - \alpha \cdot \nabla MSE(\boldsymbol{\theta})$

$\boldsymbol{\theta} := [0.5,0.2] - 0.1 \cdot [-0.1, -0.2]$

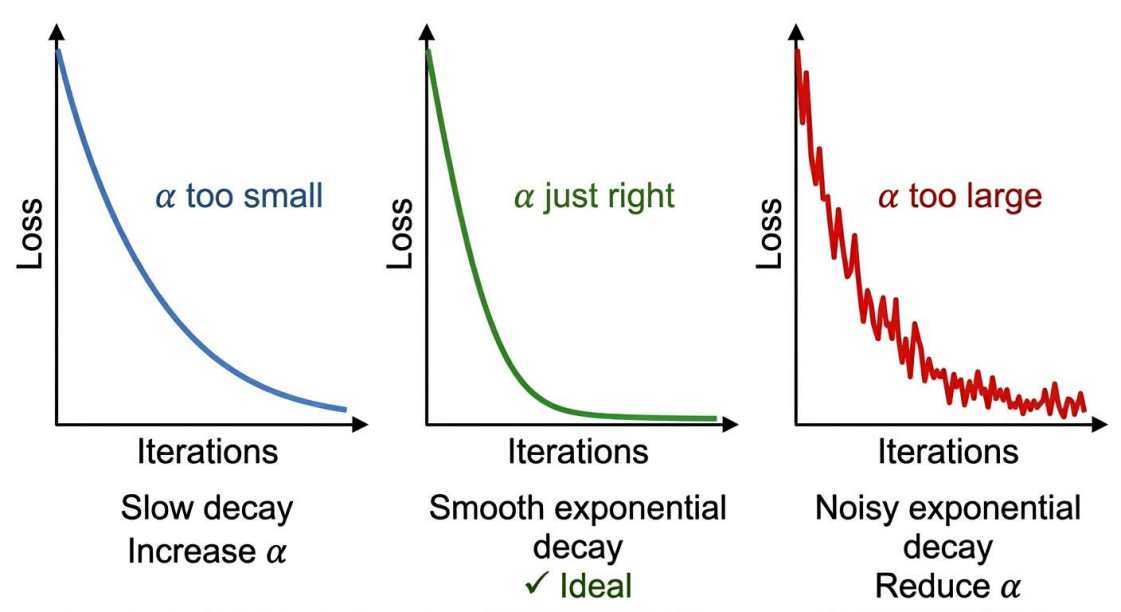
$\boldsymbol{\theta} = [0.51,0.22]$

New Prediction: $\hat{y}_2 = 0.51 \cdot 1 + 0.22 \cdot 2 = 0.95$ $\hat{y}_3 = 0.515 \cdot 1 + 0.23 \cdot 2 = 0.975$

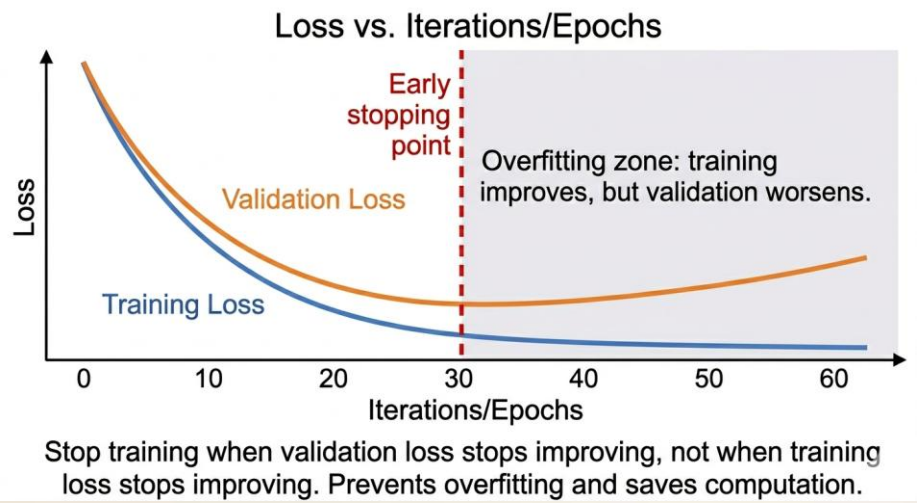
Monitoring Training: Loss Curves

$$\boldsymbol{\theta} := \boldsymbol{\theta} - \alpha \cdot \nabla MSE(\boldsymbol{\theta})$$

Learning Rate	Loss Pattern	Interpretation	Action
α too small	Very slow decrease	Creeping toward minimum	Increase α
α just right	Smooth decrease, plateaus	Converged successfully	Continue or stop
α too large	Oscillates or increases	Overshooting the minimum	Decrease α



Early Stopping: Stop training when MSE improvement stalls (change $<$ small threshold ϵ).



Gradient Descent Variants

Variant	Data Used	Updates Per Epoch	Speed	Stability	Best For
Batch	All m examples	1	Slow	Accurate	Small Datasets
Stochastic (SGD)	1 random sample	m	Fast	Noisy	Not recommended for fixed datasets
Mini-Batch	Small batch (e.g., $b = 32$)	$\frac{m}{32}$	Fast	Stable	Standard Practice

Trade-Offs

- More data per update → fewer updates per epoch, slower per-epoch progress
- Less data per update → more updates per epoch, but each update is noisier
- Mini-batch balances: ~30 updates per epoch and stable gradients

Epoch: One complete pass through all training data

Standard practice: Train for multiple epochs using mini-batch gradient descent. Each epoch: shuffle data, compute $\frac{m}{32}$ mini-batch updates, evaluate on validation set.

Training Linear Regression: Summary

The problem:

Find $\boldsymbol{\theta}$ that minimizes

$$MSE(\boldsymbol{\theta}) = \frac{1}{m}\sum_{i=1}^{m}(h_{\boldsymbol{\theta}}(\mathbf{x}_i) - \boldsymbol{y_i})^2$$

Solution 1: Normal Equation

$$\boldsymbol{\theta}^* = \left(\mathbf{X}^{\mathbf{T}}\mathbf{X}\right)^{-\mathbf{1}}\mathbf{X}^{\mathbf{T}}\mathbf{y}$$

where $\mathbf{y}$ is the target vector (actual labels for all m examples).

- Direct computation so there is no iteration.
- Impractical for large n

Solution 2: Gradient Descent

$$\boldsymbol{\theta} := \boldsymbol{\theta} - \alpha \cdot \nabla MSE(\boldsymbol{\theta})$$

- Scales to big data
- Requires tuning α

Gradient descent is the fundamental optimization algorithm in machine learning. It works for:

- Linear regression
- Logistic regression
- Deep Neural networks

Logistic Regression

Logistic Regression

Problem: Linear regression outputs real numbers (e.g., 0.91). For classification, we need probabilities or discrete classes.

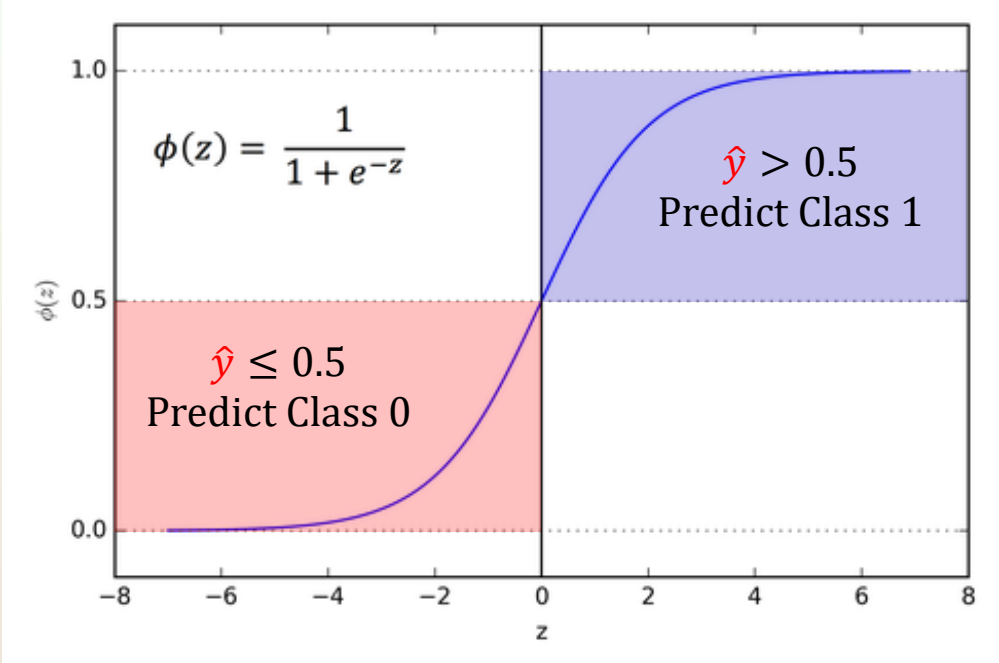
Solution: Use the ***sigmoid*** function to squash outputs to a range of $(0, 1)$: $\sigma(z) = \dfrac{1}{1 + e^{-z}}$ where $z = \boldsymbol{\theta}^{\mathbf{T}}\mathbf{x}$.

Model definition:

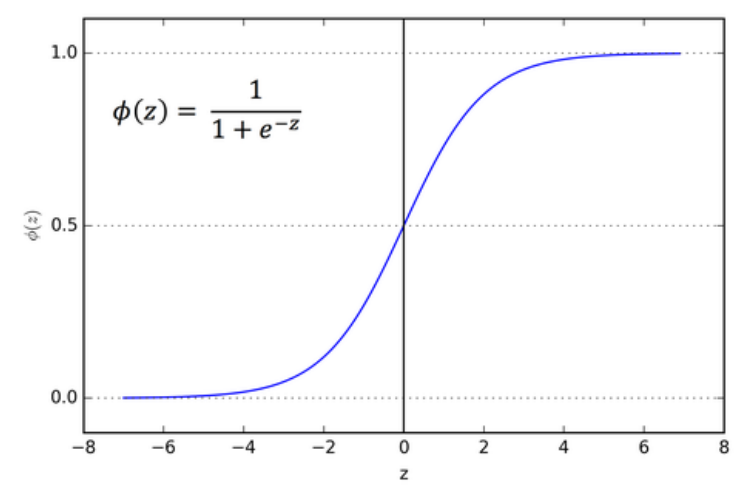
Logistic Regression = Linear model + Sigmoid activation

1. Apply sigmoid: $\hat{y} = \sigma\left(\boldsymbol{\theta}^{\mathbf{T}}\mathbf{x}\right) \in (0,1)$
2. Output: Probability for binary classification:
$$P(class = 1 \mid \mathbf{x})$$

Training: Same gradient descent as before, but with different loss function (cross-entropy instead of MSE)



Sigmoid Properties



$$\lim_{z \to -\infty} \sigma(z) = 0$$
$$\lim_{z \to \infty} \sigma(z) = 1$$

Inflection point at z = 0

Why these matter for Classification

Gives valid probabilities.

Bounded output interpretable for classification.

Fast gradient computation during training

Efficient derivative: The derivative depends only on the output value, so it can be computed once during backpropagation.

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

Clear cause-effect between inputs and predictions

Monotonically Increasing: Unambiguous relationship: as z increases, $\sigma(z)$ increases.

Flexible decision boundaries; can handle complex patterns

Non-linear S-Shape: Learns non-linear decision boundaries. Pure linearity would fail for overlapping or non-separable classes.

Loss Function for Probability Predictions

True Label = 1

Prediction	MSE Loss	Cross-Entropy Loss	MSE Interpretation	Cross-Entropy Interpretation
0.01 (very wrong)	$(1-0.01)^2 = 0.98$	$-\log 0.01 \approx 4.6$	small error	huge penalty
0.5 (also wrong)	$(1-0.5)^2 = 0.25$	$-\log 0.5 \approx 0.69$	smaller error	moderate penalty

MSE

- Both predictions look like “small errors”.
- MSE has weak gradients when most wrong: learning signal gets dampened at extremes.

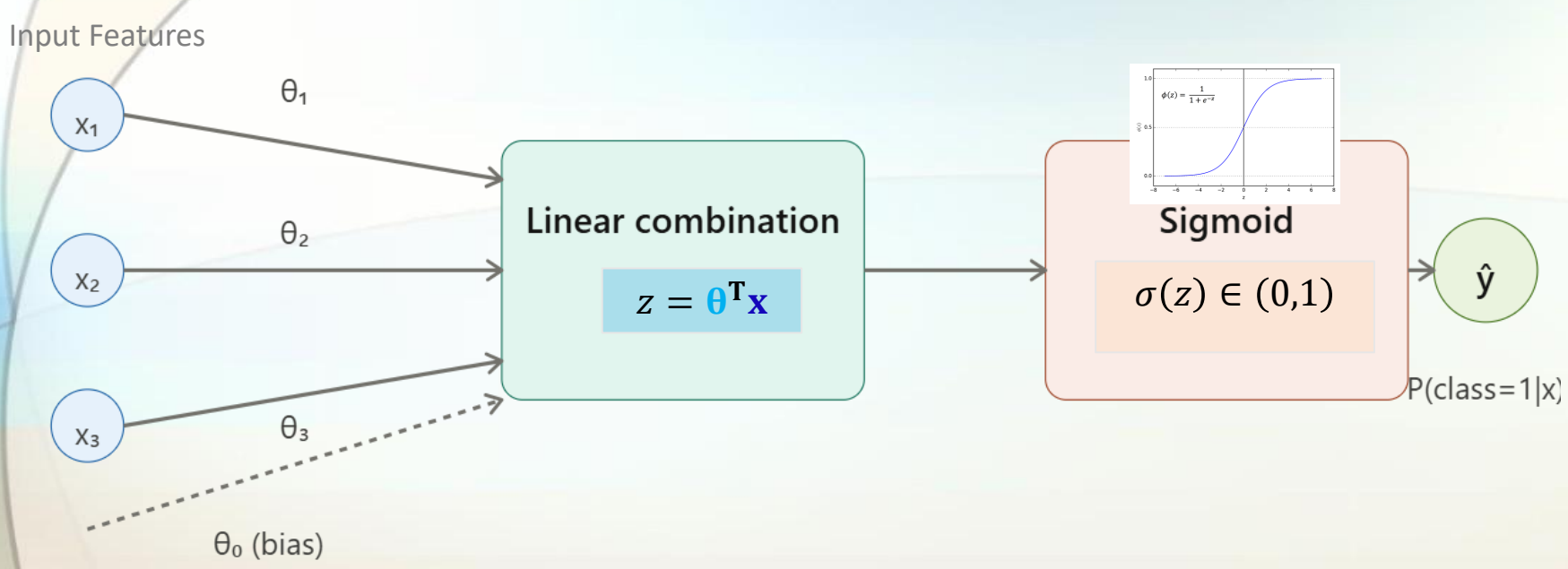
CE

- Confident mistakes (0.01) are heavily penalized.
- Strong gradients when most wrong thus it learns fast from confident mistakes.

Why CE Works?

- Sigmoid outputs are *probabilities*.
- Cross-entropy is designed to measure “how likely was the true label?” This is a natural fit for classification.

Logistic Regression is a Single-Layer NN



This is logistic regression.
It is a neural network!

Neural networks are built by stacking these layers:

- More layers = deeper networks
- Different activation functions ($ReLU$, $\tanh$)
- Same optimization principle: gradient descent with appropriate loss function

Now we can evaluate it: Precision, Recall, F1, AUC all apply to classifiers like this.